
Project Explanation: Interpaws (AI-Assisted 18-Day Sprint)

Interpaws is a full-stack web application designed to optimize scheduling and operations for veterinary clinics. It leverages AI to enhance efficiency, prevent overbooking, and provide personalized recommendations.

The MVP (Minimum Viable Product) will be built from scratch and result in a complete, tested, deployed, and demo-ready product in 18 days (November 6 – November 23, 2025).

Development Methodology: AI-Assisted Sprints

This project will be executed using a non-traditional sprint model designed to maximize focus, maintain high stimulation, and prevent burnout, explicitly accommodating an ADHD workflow and heavy AI-pairing.

Instead of long, unstructured 8-10 hour blocks, each day will be broken into 3-4 distinct, high-intensity **Sprints** followed by a **Review**.

1. **Sprint 1 (Backend/Logic):** High-focus "deep work" on a single backend or data-related task (e.g., building API endpoints, database models).
2. **Sprint 2 (Frontend/Variety):** A hard context-switch to a *different* part of the stack, typically frontend UI development. This provides the necessary variety to maintain stimulation.
3. **Sprint 3 (Integration/Problem-Solving):** A high-stimulation "problem-solving" block focused on making Sprint 1 and Sprint 2's components communicate.
4. **Daily Review & Debug (AI-Partnered):** The final block of the day. We will review all code written, perform *immediate* debugging (distributing the testing buffer), and explicitly plan the next day's Sprints.

AI's Role: The AI assistant will act as an "executive function" partner, co-pilot, and debugger. It will be used to break large tasks into specific, actionable steps; generate boilerplate code; review completed code; and help resolve bugs in real-time.

Key Components and Features (MVP):

- **Core Functionality:** Rule-based intelligent scheduling (checking staff availability, slot limits).
- **AI Integration:** Powered by **Ollama** and **sentence-transformers**. AI provides *recommendations* (e.g., best doctor/time) and *auto-scheduling suggestions* based on embeddings of client complaints, pet needs, and staff skills.
- **User Interfaces:**
 - **Admin Dashboard:** Manage schedules, surgeries, and operations.
 - **Client Portal:** Self-booking app with AI-driven suggestions.
- **Demo Simulator:** A Python (Faker) script to generate synthetic data (clients, bookings, preferences) for testing, AI simulation, and presentation.

Tech Stack:

- **Backend:** Python with FastAPI
- **Frontend:** Next.js
- **Database:** PostgreSQL with SQLAlchemy (ORM)
- **AI:** Ollama (inference), sentence-transformers (embeddings)
- **Deployment:** Docker, Vercel (Frontend), Heroku/Render (Backend)

Optimized 18-Day AI-Assisted Sprint Timeline

Day(s)	Daily Goal & Key Tasks	Key Resource (URL)
Day 1 (Nov 6)	Goal: Project Initialization & Setup. * Sprint 1 (Backend): Init FastAPI, setup <code>database.py</code> with SQLAlchemy, define core DB schemas (Clinic, Staff). * Sprint 2 (Frontend): Init Next.js, create component/page folder structure. * Sprint 3 (Deploy): Create <code>docker-compose.yml</code> to link FastAPI, Next.js, and Postgres.	FastAPI Quickstart

	* Review: Confirm all services can start and connect locally.	
Days 2-3 (Nov 7-8)	Goal: Backend Core & UI Skeletons. * Sprint 1 (Backend): Build full DB models (Pet, Client, Booking, Preferences). Build <i>rule-based</i> booking logic (e.g., <code>check_availability(time, doctor)</code>). * Sprint 2 (Frontend): Build the <i>non-functional</i> UI components for the Admin Dashboard (a calendar view, booking list) and Client Portal (booking form). * Sprint 3 (Integration): Create the first API endpoints (e.g., <code>POST /booking_rule_based</code>, <code>GET /bookings/{date}</code>). * Review: Test endpoints with a tool like Postman. Debug DB connections.	SQLAlchemy ORM Guide

Days 4-5 (Nov 9-10)	Goal: AI Integration & Client Forms. * Sprint 1 (AI/Backend): Setup Ollama endpoint. Create embedding logic (with sentence-transformers) for <i>client complaints</i> and <i>staff skills</i>. Store vectors in DB. * Sprint 2 (Frontend): Build the functional Client Portal booking form UI. * Sprint 3 (Integration): Create the AI API endpoint (e.g., <code>GET /suggest_slots</code>). The frontend client form <i>calls this endpoint</i> to get suggestions. * Review: Test AI suggestions. Are they logical? Debug embedding generation.	Sentence-Transformers Docs
--	---	-----------------------------------

Days 6-7 (Nov 11-12)	Goal: Admin Dashboard Functionality. * Sprint 1 (Backend): Build all Admin-facing CRUD endpoints (e.g., <code>UPDATE /booking/{id}</code> , <code>DELETE /booking/{id}</code> , <code>GET /all_staff</code>). * Sprint 2 (Frontend): Build the Admin UIs for managing staff, meds, and surgeries. * Sprint 3 (Integration): Connect the Admin Dashboard UIs to the backend APIs. Ensure the admin can create, view, update, and delete bookings from the UI. * Review: Full test of all Admin CRUD operations.	Next.js Data Fetching
-------------------------------------	--	-----------------------

Days 8-9 (Nov 13-14)	Goal: Client Portal Full Flow. * Sprint 1 (Backend): Finalize all client-facing endpoints (e.g., POST /self_book, GET /my_bookings, POST /my_preferences). * Sprint 2 (Frontend): Finalize the client portal: add preference logging, "My Bookings" page. * Sprint 3 (Integration): Test the <i>full end-to-end client flow</i> : (Client lands -> gets AI suggestion -> books slot -> sees it in "My Bookings"). * Review: Debug the entire client-side user journey.	Next.js Forms Guide
-------------------------------------	---	---------------------

Days 10-11 (Nov 15-16)	Goal: Simulator & AI Feedback Loop. * Sprint 1 (Sim/Data): Heavily expand the Faker simulator script. Generate 100s of realistic clients, pets, bookings, and <i>complaint/preference strings</i> . * Sprint 2 (AI/Backend): Build the simple "feedback loop" (e.g., an endpoint that logs successful booking details to a new table for future AI analysis). * Sprint 3 (Integration): <i>Run the simulator</i> to fully populate the database. Test the AI recommendations against this new, rich data. * Review: Are the AI suggestions <i>better</i> now? Debug the simulator script.	Faker Python Docs
---	--	----------------------

Days 12-14 (Nov 17-19)	Goal: Feature Freeze, Polish & Integration. * Sprint 1 (Bug Hunt): Full E2E test of the <i>Admin</i> workflow. Find and list all bugs. * Sprint 2 (Bug Hunt): Full E2E test of the <i>Client</i> workflow. Find and list all bugs. * Sprint 3 (Polish & Fix): Fix the highest priority bugs. Add UI polish: loading spinners, error messages, and ensure consistent styling. * Review: Triage remaining bugs. <i>No new features are allowed after Day 14.</i>	Pytest Tutorial
---	---	-----------------

Days 15-16 (Nov 20-21)	Goal: Deployment & Live Testing. * Sprint 1 (Deploy): Finalize Dockerfiles and <code>docker-compose.yml</code> . Provision a cloud PostgreSQL database. * Sprint 2 (Deploy): Deploy Backend (e.g., Render/Heroku) and Frontend (Vercel). Configure all environment variables. * Sprint 3 (Test): Test the <i>live, deployed application</i> . This is critical for finding new bugs (e.g., CORS, DB permissions). * Review: Fix deployment-specific bugs.	Vercel Deployment
---	--	----------------------

Days 17-18 (Nov 22-23)	Goal: Demo Prep & Final Buffer. * Sprint 1 (Demo Script): Write a step-by-step presentation script. (e.g., "1. Show admin dash. 2. As client, book for 'limping dog'. 3..."). * Sprint 2 (Demo Data): <i>Wipe the live DB</i> and run the simulator script <i>one last time</i> to create a clean, perfect dataset for the demo. * Sprint 3 (Practice): <i>Record</i> yourself performing the demo script. Ensure all features work as expected. * Review: <i>Final emergency bug fixes only.</i> The product is complete.	Docker Compose Guide
--	--	-----------------------------